

Nama : Shananda Putri Aisyah

NIM : 245150601111013

Perbandingan Imperatif dan Deklaratif dalam Perubahan Teks

2. Objek yang diubah secara langsung adalah TextView dengan ID tvMessage.
Baris kode: `tvMessage.text = "Tombol sudah diklik"`
3. Nilai dari variabel state bernama message.
Baris kode: `message = "Tombol sudah diklik"`
4. Karena Jetpack Compose menggunakan pendekatan deklaratif ($UI = f(state)$). Kita tidak memegang referensi komponen UI secara langsung, melainkan mendeklarasikan tampilan berdasarkan state. Ketika state berubah, Compose secara otomatis memperbarui bagian UI yang terdampak melalui recomposition.
5. Pengguna memicu suatu aksi (event, misal klik tombol) $\rightarrow$ program mencari referensi komponen View secara manual di memori (misalnya via `findViewById`) $\rightarrow$ program mengubah properti komponen tersebut secara langsung (perubahan tampilan).
6. Pengguna memicu aksi (event) $\rightarrow$ aksi tersebut mengubah nilai dari suatu state $\rightarrow$ Compose mendeteksi perubahan state dan memicu proses recomposition (menggambar ulang fungsi composable) $\rightarrow$ perubahan tampilan terjadi secara otomatis di layar.
7. Tampilan teks di layar tidak akan berubah (tetap "Belum diklik").
Penyebab: Variabel biasa bukan merupakan observable state, sehingga perubahannya tidak dipantau oleh Compose dan tidak memicu proses recomposition.
8. Berfokus pada langkah-langkah prosedural bagaimanakah cara mengubah UI secara manual (cari komponennya, lalu ubah propertinya satu per satu).
Deklaratif: Berfokus pada seperti apa wujud UI berdasarkan kondisi data saat ini; cukup ubah datanya (state), dan framework yang akan mengurus pembaruan tampilannya secara otomatis.

Event Handling State

2. Objek yang diubah secara langsung adalah TextView dengan ID tvMessage.
Baris kode: `tvMessage.text = "Tombol sudah diklik"`
3. Nilai dari variabel state bernama message.
Baris kode: `message = "Tombol sudah diklik"`

4. Karena Jetpack Compose menggunakan pendekatan deklaratif ($UI = f(state)$). Kita tidak memegang referensi komponen UI secara langsung, melainkan mendeklarasikan tampilan berdasarkan state. Ketika state berubah, Compose secara otomatis memperbarui bagian UI yang terdampak melalui recomposition.
5. Pengguna memicu suatu aksi (event, misal klik tombol) $\rightarrow$ program mencari referensi komponen View secara manual di memori (misalnya via `findViewById`) $\rightarrow$ program mengubah properti komponen tersebut secara langsung (perubahan tampilan).
6. Pengguna memicu aksi (event) $\rightarrow$ aksi tersebut mengubah nilai dari suatu state $\rightarrow$ Compose mendeteksi perubahan state dan memicu proses recomposition (menggambar ulang fungsi composable) $\rightarrow$ perubahan tampilan terjadi secara otomatis di layar.
7. Tampilan teks di layar tidak akan berubah (tetap "Belum diklik").
Penyebab: Variabel biasa bukan merupakan observable state, sehingga perubahannya tidak dipantau oleh Compose dan tidak memicu proses recomposition.
8. Imperatif: Berfokus pada langkah-langkah prosedural bagaimanakah cara mengubah UI secara manual (cari komponennya, lalu ubah propertinya satu per satu).
Deklaratif: Berfokus pada seperti apa wujud UI berdasarkan kondisi data saat ini; cukup ubah datanya (state), dan framework yang akan mengurus pembaruan tampilannya secara otomatis.

Event Handling State

2. Muncul log sesuai kode, contohnya: Counter berubah 1, Counter berubah 2, dan seterusnya.
3. Tampilan angka pada teks (Counter: 0) tidak berubah sama sekali dan tetap bernilai 0.
4. Karena variabel count dideklarasikan sebagai variabel lokal biasa, bukan sebagai state, sehingga perubahannya tidak memberitahu Compose untuk menggambar ulang UI.
9. Logcat tetap menampilkan pesan perubahan nilai counter (Counter berubah 1, Counter berubah 2, dst.).
10. Angka pada teks di layar ikut bertambah dan berubah secara real-time.

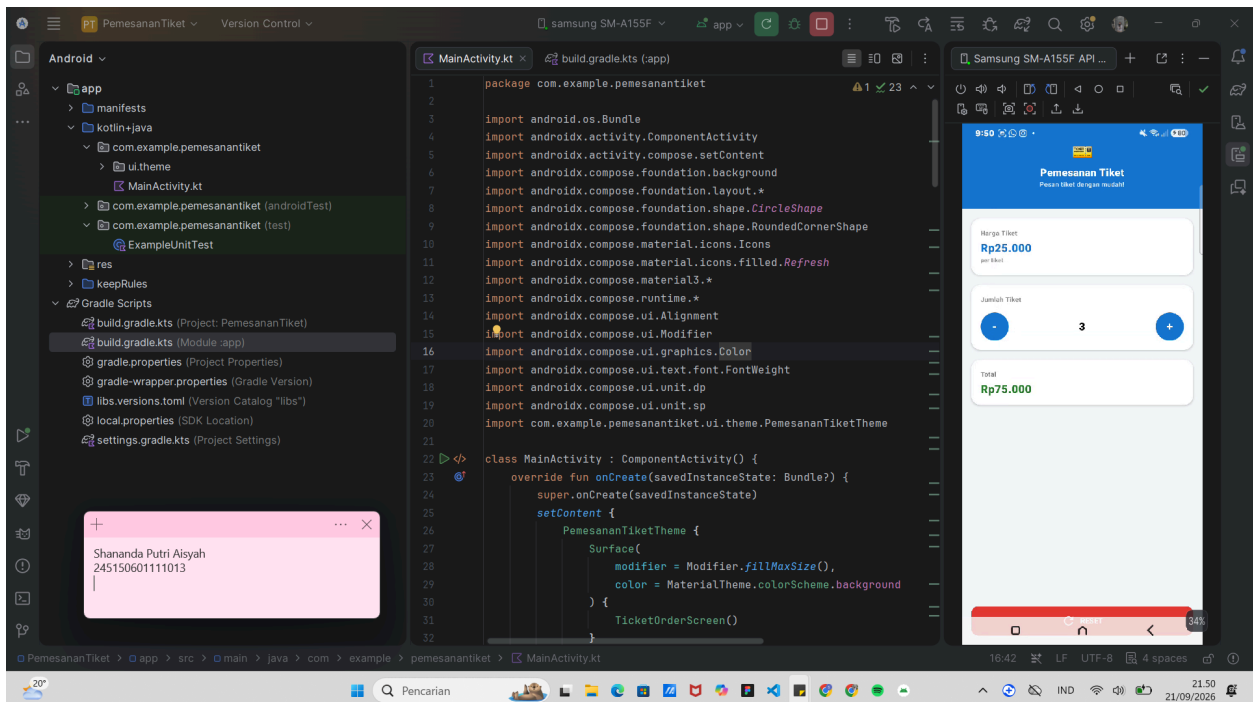
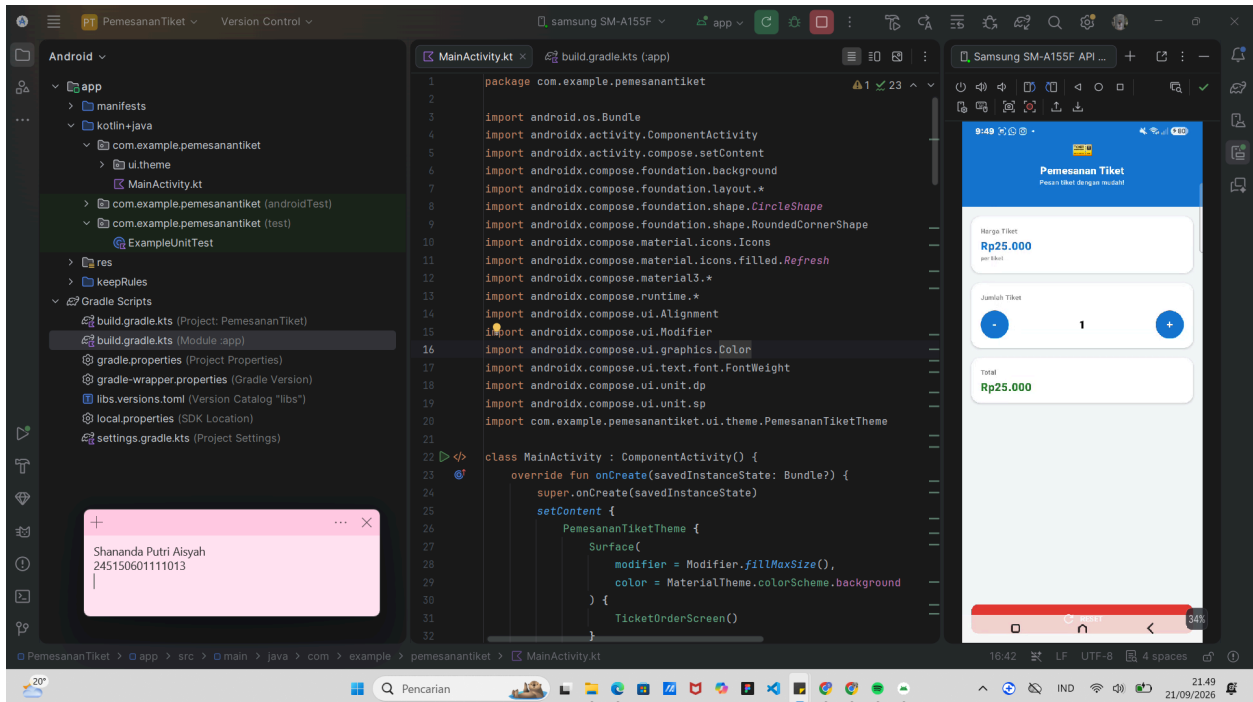
11. Karena penggunaan `mutableStateOf` yang digabungkan dengan `remember` menciptakan state yang dapat diamati (`observable`), sehingga setiap nilainya berubah, Compose mendeteksinya dan melakukan recomposition.
12. Proses di mana Jetpack Compose menjalankan ulang fungsi composable secara otomatis ketika ada state yang dibacanya mengalami perubahan, guna memperbarui tampilan UI di layar.
15. Muncul log dengan tag TEST yang menampilkan karakter atau teks yang diinputkan, contohnya: Text berubah: a, Text berubah: ab, dst.
16. Karakter yang diketik langsung muncul dan tampil di dalam kotak `OutlinedTextField`.
17. Event `onValueChange` menangkap setiap input dan memperbarui nilai state text, yang kemudian memicu recomposition sehingga UI menampilkan teks terbaru.
18. State `count` (untuk menyimpan angka penghitung) dan state `text` (untuk menyimpan nilai string pada `TextField`).
19. Event `onClick` pada Button dan event `onValueChange` pada `OutlinedTextField`.
20. `onClick` dijalankan saat pengguna menekan komponen Tombol.
`onValueChange` dijalankan setiap kali ada interaksi pengetikan atau penghapusan karakter di dalam `OutlinedTextField`.

Implisit Intent

11.
 - a. `Intent.ACTION_VIEW`: Menampilkan data tertentu kepada pengguna, misalnya membuka halaman web melalui URL di browser atau menampilkan lokasi di peta.
 - b. `Intent.ACTION_DIAL`: Membuka aplikasi panggilan telepon (dialer) dengan menyertakan nomor tujuan agar pengguna bisa langsung menekan tombol panggil.
 - c. `Intent.ACTION_SENDTO`: Mengirim pesan atau email langsung ke penerima tertentu (paling sering digunakan dengan skema `mailto:`).
16. Objek data (dalam bentuk Uri) berfungsi untuk membawa informasi spesifik atau target sasaran yang akan diproses oleh Action yang dipanggil (seperti alamat URL, nomor telepon, koordinat GPS, atau alamat email).
17. Sistem Android menggunakan kombinasi antara Action dan Data untuk melakukan proses pencocokan (`Intent Resolution`). Sistem akan menyaring daftar aplikasi yang terpasang di perangkat dan hanya menampilkan

aplikasi-aplikasi yang memiliki kemampuan (intent filter) untuk menangani kombinasi aksi dan tipe data tersebut.

TUGAS PRAKTIK



Link Github : <https://github.com/Shanandaputri/PemesananTiket.git>

